

CSCE 649 PHYSICALLY-BASED MODELING · MODULE 05 · LECTURE NOTES

Collision Detection

Finding out what is touching what, fast enough.

MODULE OUTCOMES

- Design a broad phase and justify the structure chosen.
- Implement the separating axis test and explain GJK.
- Distinguish discrete from continuous detection and explain tunnelling.
- Generate contact manifolds rather than single points.
- Handle self-collision for deformable objects.

1 Broad phase

THE ARITHMETIC THAT FORCES A BROAD PHASE

Ten thousand objects give roughly 5×10^7 pairs. At 60 frames per second that is 3×10^9 pair tests per second — before any of them does useful work.

And almost every one of those pairs is nowhere near touching. The great majority of the computation is spent establishing that objects on opposite sides of the level are not in contact.

The broad phase eliminates them cheaply using bounding volumes — usually axis-aligned boxes — and passes a small candidate list to the exact narrow-phase test.

This is structurally identical to the acceleration structure argument in CSCE 647 Module 04: a cheap conservative test that allows the overwhelming majority of work to be skipped. The pattern appears again in Module 10 of that course, for volume majorants, and it is worth recognising as a general technique rather than a domain-specific trick.

Structure	Suits	Weakness
Uniform grid	Many objects of similar size in a bounded domain. Insertion and query are $O(1)$.	Objects much larger than a cell occupy many cells; objects much smaller waste them. Size variety is fatal.
Spatial hash	An unbounded domain, by hashing cell coordinates rather than storing a grid.	Hash collisions cause false candidate pairs; memory access is scattered, which hurts more than the asymptotic analysis suggests.
Sweep and prune	Coherent motion. Keep interval endpoints sorted along each axis; overlapping intervals on all axes are candidate pairs.	Degenerates when everything moves at once, and performs poorly when objects cluster densely along the sweep axis.

Structure	Suits	Weakness
Dynamic BVH	Mixed object sizes and general scenes. The usual answer.	Requires refitting or periodic rebuilding as objects move.
Octree / kd-tree	Largely static geometry.	Poorly suited to moving objects — the same reason kd-trees lost to BVHs in rendering.

COHERENCE IS THE LEVER

At 60 Hz, almost nothing moves far between frames. **The set of overlapping pairs this frame is nearly the set from last frame.** A broad phase that recomputes from scratch is discarding that information.

Sweep and prune exploits it directly: the sorted order of endpoints barely changes, so an insertion sort — which is $O(n)$ on nearly-sorted input — restores it, and the pairs that enter or leave the overlap set can be tracked incrementally.

A dynamic BVH refits rather than rebuilding: update the leaf bounds and propagate upward, which is $O(n)$ and preserves the tree topology. Quality degrades slowly as objects move apart, so subtrees are rebuilt selectively when a quality metric falls too far.

Fatten the bounds by a margin. If an object's stored bound is larger than the object by some slack, the object can move within that slack without the structure needing any update at all. This single trick removes most of the update cost, and every engine does it.

2 Narrow phase

THE SEPARATING AXIS THEOREM

Two convex shapes are disjoint if and only if there exists an axis onto which their projections do not overlap. Such an axis is called a separating axis.

For convex polyhedra, only a finite set of candidate axes need be tested: the face normals of both shapes, and the cross products of every pair of edges, one from each. For two boxes that is 15 axes.

Finding a separating axis proves disjointness immediately, so the test can return as soon as one is found — which makes the common case, where objects are far apart, very cheap.

Finding none proves the shapes overlap, and the axis with the smallest overlap gives you the **contact normal** and the **penetration depth** directly, which is exactly what collision response (Module 06) needs. The test produces the answer and the response data together.

Method	Applies to	Produces
Analytic tests	Sphere–sphere, sphere–plane, capsule–capsule.	Exact answers in a few operations. Always special-case these.
SAT	Convex polyhedra.	Overlap or not; normal and depth when overlapping.

Method	Applies to	Produces
GJK	Any convex shape with a support function.	Distance and closest points when separated. Returns only 'overlapping' when they intersect.
EPA	GJK's companion, run when GJK reports overlap.	Penetration depth and normal, by expanding a polytope within the Minkowski difference.
Triangle–triangle	Arbitrary meshes.	The workhorse for deformables and cloth , where nothing is convex.

GJK OPERATES ON THE MINKOWSKI DIFFERENCE

The Minkowski difference of two sets is the set of all differences between a point in one and a point in the other. **Two shapes intersect if and only if their Minkowski difference contains the origin**, which converts an intersection question into a point-containment question.

GJK never constructs the difference, which would be prohibitively expensive. It requires only a **support function**: given a direction, return the furthest point of the shape in that direction. The support function of the difference is the difference of the support functions.

The algorithm iteratively builds a simplex — up to a tetrahedron — of points in the Minkowski difference, attempting to enclose the origin, and converges in a handful of iterations.

Because it needs only support functions, it handles any convex shape whatsoever: boxes, spheres, capsules, cylinders, cones, convex hulls of point sets, and shapes defined only implicitly. That generality, more than its speed, is why it is in every physics engine.

3 The time dimension

TUNNELLING

Discrete collision detection tests for overlap at the end of each time step. A fast-moving object can be entirely in front of a thin wall at step n and entirely behind it at step $n+1$.

At no tested instant do the shapes overlap, so no collision is ever detected. The bullet passes through the wall.

This is not a bug in the intersection test. The test gave the correct answer at both instants. The failure is that the continuous motion between those instants was never considered at all.

It is worst for small fast objects against thin geometry — which describes bullets and walls precisely, and is why the feature is labelled 'bullet' or 'continuous' in most engines.

	Discrete (a posteriori)	Continuous (a priori)
Method	Advance the state, then test for overlap and resolve any that is found.	Find the earliest time of contact within the step, advance to exactly that time, resolve, and continue.
Cost	Cheap. One test per candidate pair per step.	Much more expensive. Requires root finding on a function of time, per pair.
Tunnelling	Yes.	No.
Penetration	Objects interpenetrate and must be pushed apart, which can visibly pop.	Never occurs.
Failure mode	Tunnelling; pops during recovery.	Can deadlock: in dense contact, each resolution creates another contact at an earlier time and the step never completes.
Use	The default for essentially everything.	Selectively, for objects flagged as fast.

- **Substepping.** Smaller steps mean less motion per step and proportionally less tunnelling. Often sufficient, and far simpler than CCD.
- **Speculative contacts.** Expand each object's bounds by the distance it will travel this step, and create contact constraints for collisions that *might* occur. The solver then prevents the penetration before it happens. Cheap, robust, and underrated — most of CCD's benefit for a fraction of the complexity.
- **Swept bounds in the broad phase only.** Use the union of the start and end bounds to catch the candidate pair, then run a discrete narrow phase. Catches the pair without the cost of a continuous exact test.
- **Ray casting for small fast objects.** A bullet has negligible extent; modelling it as a ray rather than a rigid body is both cheaper and more correct.
- **Flag objects individually.** Running CCD on every object is unaffordable and unnecessary — almost nothing in a scene moves fast enough to tunnel.

4 Manifolds

A SINGLE CONTACT POINT IS NOT ENOUGH

A box resting flat on the ground is in contact along an entire face, not at one point.

Given a single contact point, the solver can only resist rotation about that point, so the box rocks. Next frame a different point is found, and it rocks the other way. The result is jitter, and in a stack the jitter compounds until the stack falls.

A contact manifold is the full set of contact points between two shapes — typically reduced to at most four for a face-face contact, chosen to maximise the contact area they enclose. Four points are enough to resist rotation about every axis.

Persist manifolds between frames and match contact points by feature identity (which face, which edge, which vertex), not by position. This enables the warm starting of Module 04, where each contact's accumulated impulse carries over, and **it is what makes stable stacking possible at all**. A stack of ten boxes standing for sixty seconds — Project 1's requirement — is a test of the manifold machinery more than of the solver.

5 Self-collision

- **The broad phase must operate within a single object.** For cloth, every triangle can potentially contact every other triangle of the same mesh, so a BVH over the object's own triangles is required — and it must be refitted every step, because the geometry deforms.
- **Adjacent triangles must be excluded explicitly.** Triangles sharing a vertex or an edge always 'intersect' by any geometric test, so topological adjacency has to be checked and skipped. Forgetting this produces a cloth that immediately locks solid.
- **Two cases are needed: vertex–triangle and edge–edge.** Checking only vertex–triangle misses configurations where two edges cross without any vertex being inside a triangle — which is exactly what happens when cloth folds.
- **Once cloth has interpenetrated, it rarely recovers.** There is no local information indicating which way to separate, and pushing apart arbitrarily tends to make the tangle worse. **Prevention is genuinely necessary here**, which is why cloth justifies continuous detection where rigid bodies do not.
- **Give cloth a thickness.** Treat each triangle as having a small offset volume and separate surfaces before they touch, rather than after they overlap. This is Bridson's approach and it is what makes production cloth robust.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Ericson — Real-Time Collision Detection	The standard reference for this entire module. The author's errata and blog are free and substantial.
Gino van den Bergen — GJK materials (free)	GJK and its variants from someone who has implemented them repeatedly. The clearest free explanation of §2.
Erin Catto — GDC talks on contact manifolds and continuous collision (free)	The manifold persistence of §4 and speculative contacts of §3, from production experience.
Bridson, Fedkiw & Anderson — Robust Treatment of Collisions, Contact and Friction for Cloth (free)	The self-collision approach of §5, including cloth thickness. The standard reference.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement brute-force n^2 broad phase and a uniform grid. Plot query time against object count for both and find the crossover.
- 2 Implement sweep and prune. Measure the cost per frame with objects moving slowly and with objects moving randomly, and explain the difference.
- 3 Implement a dynamic BVH with refitting and bound fattening. Report how the fattening margin trades update cost against candidate pair count.
- 4 Implement SAT for boxes. Verify that the minimum-overlap axis gives a sensible contact normal in a variety of configurations.
- 5 Implement GJK with support functions for a box, a sphere, and a cylinder. Verify distances against analytic values where available.
- 6 Demonstrate tunnelling: fire a small sphere at a thin wall at increasing speeds and find the speed at which it passes through.
- 7 Implement speculative contacts and show that the same test no longer tunnels. Report the cost.
- 8 Implement manifold generation for box–box contact. Show that a box rests stably with four contact points and rocks with one.
- 9 Build the ten-box stack from Project 1 and measure how long it stands with and without manifold persistence.
- 10 Implement cloth self-collision with both vertex–triangle and edge–edge tests. Remove the edge–edge test and demonstrate a configuration it misses.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is an n^2 broad phase infeasible, and what does the broad phase replace it with?
- 2 Compare four broad-phase structures and say when each is appropriate.
- 3 What is coherence, and name three ways a broad phase exploits it?
- 4 State the separating axis theorem and say what SAT produces besides a yes/no answer.
- 5 What does GJK need from a shape, and why does that make it general?
- 6 Explain tunnelling and say why it is not a bug in the intersection test.
- 7 Give four alternatives to full continuous collision detection.
- 8 Why does a single contact point make a resting box rock, and what is a manifold?
- 9 Give three things that make cloth self-collision harder than object-object collision.